

Unit IV Classification and Prediction

Issues Regarding Classification and Prediction – Classification by Decision Tree Introduction – Bayesian Classification – Rule Based Classification – Classification by Back propagation – Support Vector Machines – Associative Classification – Lazy Learners – Other Classification Methods – Prediction – Accuracy and Error Measures – Evaluating the Accuracy of a Classifier or Predictor – Ensemble Methods – Model Section.

Issues Regarding Classification and Prediction –

Classification and Prediction Issues

- **Data Cleaning:** Data cleaning involves removing the noise and treatment of missing values. ...
- **Relevance Analysis:** The database may also have irrelevant attributes. ...
- **Data Transformation and reduction:** The data can be transformed by any of the following methods.
- **Data cleaning** – This defines the pre-processing of data to eliminate or reduce noise by using smoothing methods and the operation of missing values (e.g., by restoring a missing value with the most generally appearing value for that attribute, or with the best probable value established on statistics). Although various classification algorithms have some structures for managing noisy or missing information, this step can support reducing confusion during learning.
- **Relevance analysis** – There are various attributes in the data that can be irrelevant to the classification or prediction task. For instance, data recording the day of the week on which a bank loan software was filled is improbable to be relevant to the success of the software. Moreover, some different attributes can be redundant.

Therefore, relevance analysis can be implemented on the data to delete some irrelevant or redundant attributes from the learning procedure. In machine learning, this step is referred to as feature selection. It contains such attributes that can otherwise slow down, and likely mislead the learning step.

Correctly, the time used on relevance analysis, when inserted to the time used on learning from the resulting “reduced” feature subset, and must be less than the time that would have been used on learning from the initial set of features. Therefore, such analysis can help boost classification effectiveness and scalability.

- **Data transformation** – The data can be generalized to a larger-level approach. Concept hierarchies can be used for these goals. This is especially helpful for continuous-valued attributes. For instance, mathematical values for the attribute income can be generalized to the discrete field including low, medium, and high. Likewise, nominal-valued attributes, such as the street, can be generalized to larger-level concepts, such as the city.

Because generalization shortens the initial training data, fewer input/output operations can be included during learning. The data can also be normalized, especially when neural networks or techniques containing distance measurements are used in the learning step.

Normalization includes scaling all values for a given attribute so that they decline inside a small specified area, including -1.0 to 1.0, or 0 to 1.0. In these approaches that apply distance measurements, for instance, this can avoid attributes with originally high ranges (such as, income) from

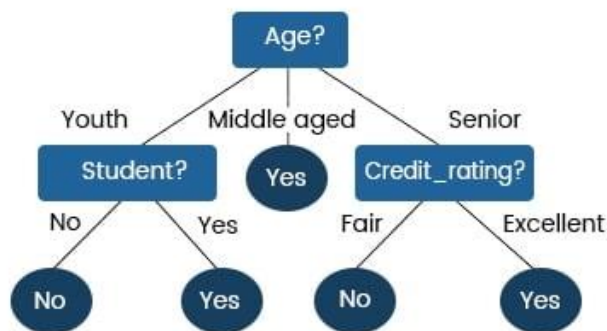
Classification by Decision Tree Introduction –

A decision tree is a class discriminator that recursively partitions the training set until each partition consists entirely or dominantly of examples from one class. Each non-leaf node of the tree contains a split point that is a test on one or more attributes and determines how the data is partitioned.

Decision Tree Induction

In data mining, the supervised learning technique of decision tree is utilized for classification and regression. There's a tree there that aids with selecting choices. Tree-like models for classification or deterioration can be generated using the decision tree. While creating the decision tree, the data set is partitioned into smaller subgroups. As a result, we get a decision tree with leaf nodes upon completion. At the very least, a decision node will have two forks. The leaf nodes represent classification or a final verdict. Leaf nodes are the topmost decision nodes in a tree and are connected to the best predictor, also known as the root node. Decision trees are flexible enough to handle both numerical and categorical information.

Decision Tree in Data Mining



A decision tree for the concept buys_computer, indicating whether a customer at AllElectronics is likely to purchase a computer. Each internal (nonleaf) node represents a test on an attribute. Each leaf node represents a class (either buys_computer = yes or buys_computer = no).

Depicts the structure of a typical data mining decision tree. It stands for the idea of buying a computer and can foretell whether or not an All Electronics consumer will make that purchase.

Ovals represent leaf nodes, while rectangles represent internal nodes. Not all decision tree algorithms can generate non-binary trees; some can only generate binary trees (in which each internal node branches to precisely two other nodes).

When asked, "*How are decision trees used for categorizing objects?*" The attribute values of a given tuple (X) are compared to those in the decision tree to determine whether or not the tuple belongs to a specific class. The data is followed up from the root via a series of intermediate nodes until it reaches a leaf node that stores the class prediction for that tuple. Classification rules may be generated using decision trees with little effort.

Why are decision trees used in so many classification systems? Decision tree classifier design is well-suited to discovery through exploration since it does not need prior knowledge of the domain or the establishment of parameters. High-dimensional data is no problem for decision trees. The tree structure they use to display learned information is natural and straightforward to understand for most people. Decision tree induction's learning and categorization processes are quick and easy.

In most cases, classification decisions made using decision trees are highly accurate. However, the quality of the available data may influence the outcome. For categorization purposes, several fields have turned to decision tree induction algorithms, including those in medicine, industry, economics, astronomy, and molecular biology.

Algorithm For Decision Tree

Algorithm: Generate a decision tree. Generate a decision tree from the training tuples of data partition D.

Input:

- 1) Data partition, D, which is a set of training tuples and their associated class labels;
- 2) Attribute list, the set of candidate attributes;
- 3) Attribute selection method, a procedure to determine the splitting criterion that “best” partitions the data tuples into individual classes. This criterion consists of a splitting attribute and, possibly, a split point or splitting subset.

Output: A decision tree.

Method:

- **(Step 1)** Create a node N;
- **(Step 2)** If tuples in D are all of the same class, C then
- **(Step 3)** Return N as a leaf node labeled with the class C;
- **(Step 4)** If attribute list is empty then
- **(Step 5)** Return N as a leaf node labeled with the majority class in D; // majority voting

- **(Step 6)** Apply Attribute selection method(D , attribute list) to find the “best” splitting criterion;
- **(Step 7)** Label node N with splitting criterion;
- **(Step 8)** If splitting attribute is discrete-valued and multiway splits allowed, then // not restricted to binary trees
- **(Step 9)** Attribute list $\leftarrow$ attribute list – splitting attribute; // remove splitting attribute
- **(Step 10)** For each outcome j of splitting criterion // partition the tuples and grow subtrees for each partition
- **(Step 11)** Let D_j be the set of data tuples in D satisfying outcome j ; // a partition
- **(Step 12)** If D_j is empty then
- **(Step 13)** Attach a leaf labeled with the majority class in D to node N ;
- **(Step 14)** Else attach the node returned by Generate decision tree(D_j , attribute list) to node N ;
end for
- **(Step 15)** Return N ;

4) There are three inputs to the algorithm:

The first one is dimension (D), a list of attributes, and a technique for selecting those attributes (Attribute selection method). D is shorthand for “*data partition*,” which is the official name for this concept. At first, it is the full collection of training tuples and their corresponding class labels. The set of properties that characterize the tuples is the parameter attribute list. The attribute selection technique This method uses a metric for selecting attributes, like the Gini index or information gain. In most cases, the attribute selection measure determines whether or not the tree contains only binary nodes. The Gini index is one such attribute selection measure that requires the output tree to be binary. While certain factors limit branching, such as knowledge acquisition, others do not (i.e., two or more branches to be grown from a node).

5) The training tuples in D are initially represented by a single node in the tree, denoted by the letter N . (*step 1*).

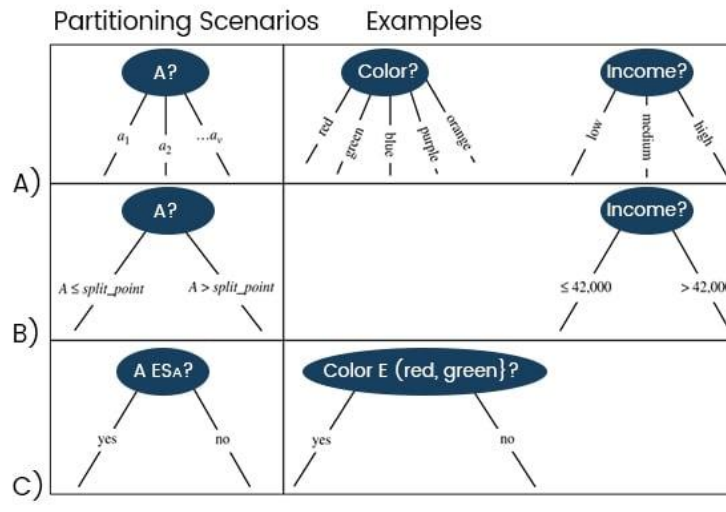
6) When node N is a leaf and is assigned a class based on whether or not the tuples in D are all of the same class, we say that D is a class-based tree (*steps 2 and 3*). It's important to realize that 4 and 5 are dead ends. The algorithm's last section explains each of the possible exit situations.

7) In this case, the algorithm uses the Attribute selection approach to choose the criteria for the split. By identifying the “optimal” way to partition the tuples in D into individual classes, realize the splitting criteria indicate which attribute should be tested at node N . (*step 6*). With respect to the test results, the splitting criterion also specifies which branches to develop from node N . In particular, the splitting criterion denotes the splitting attribute and, optionally, a split-point or splitting subset. The splitting

criteria are established to produce "pure" partitions at each branch. Pure partitions contain only tuples from the same category. In other words, we want the partitions that arise from splitting the tuples in D following the mutually exclusive results of the splitting criterion to be as clean as feasible.

8) The splitting criterion is a test at attributed is indicated by the label (*step 7*). For each possible result of the splitting criterion, node N sprouts a new branch. Tuples in D are separated as needed (*step 1* indicates, there are three distinct outcomes available if we define A as the dividing characteristic. According to the data in the training set, the value of A can take on v different forms: $a_1, a_2, \dots, a_v$.

- If A has a finite number of values, then the test results at node N will map directly onto those values. A new branch is generated and labeled with the value a_j for each value a_j of A . The subset of class-labeled tuples in D with the value a_j of A is denoted by the partition D_j . Because all tuples within a particular partition have the same value for A , A is no longer relevant to the partitioning process. As a result, it has been taken off the list of attributes (*steps 8 to 9*).
- If A is continuous, then there are two possible test results at node N : either A is less than or greater than the split point. When the Attribute selection method returns a split point as part of the splitting criterion, use that point as the split point. (In practice, the split-point, a , is commonly selected as the midpoint of two known adjacent values of A and may not be a pre-existing value of A from the training data.) Based on these findings, two new branches are developed from N and given appropriate labels. D_1 contains the class-labeled tuples in D for which A split point and D_2 contains the remaining tuples.
- The value of A is discrete. Hence a binary tree must be constructed (per the requirements of the attribute selection measure or method being used): At node N , we do an " $A \leq a$?" test. When Attribute selection is used as the splitting criterion, it returns SA as the subset of A that may be divided into smaller pieces. This list contains only some of the A values currently known to exist. The test at node N is considered to be true if and only if the provided tuple has a value a_j of A and $a_j \in SA$. Shows that N is the parent of two separate branches. D_1 is the subset of class-labeled tuples in D that pass the test, and it corresponds to the left branch out of N , which is always labeled yes. D_2 is the subset of class-labeled tuples from D that fail the test because they stem from the right branch of N , labeled no.



9) The algorithm uses the same process recursively to form a decision tree for the tuples at each resulting partition, D_j , of D (step 14).

The recursive partitioning stops only when any of the following terminating conditions is true:

- All tuples in partition D (represented at node N) belong to the same class (steps 2 and 3),
- There are no remaining attributes on which the tuples may be further partitioned (step 4). In this case, majority voting is employed (step 5). This involves converting node N into a leaf and labeling it with the most common class in D . Alternatively, the class distribution of the node tuples may be stored.
- There are no tuples for a given branch; that is, a partition D_j is empty (step 12). In this case, a leaf is created with the majority class in D (step 13).

10) The resulting decision tree is returned (step 15).

Advantages of Using Decision Trees:

There are few advantages of using decision tree induction in data mining. Let's explore these advantages:

- Information scaling is unnecessary when using a decision tree.
- Data with missing values do not hinder the decision tree construction process either.
- Automatic and straightforward to explain to both the technical team and the stakeholders, a decision tree model can do the heavy lifting for you.
- Decision trees need less work in pre-processing data than other methods.
- To use a decision tree, it is not necessary to normalize the information being analyzed.

Bayesian Classification –

Bayesian classification is based on Bayes' Theorem. Bayesian classifiers are the statistical classifiers. Bayesian classifiers can predict class membership probabilities such as the probability that a given tuple belongs to a particular class.

Baye's Theorem

Bayes' Theorem is named after Thomas Bayes. There are two types of probabilities –

- Posterior Probability $[P(H/X)]$
- Prior Probability $[P(H)]$

where X is data tuple and H is some hypothesis.

According to Bayes' Theorem,

$$P(H/X) = P(X/H)P(H) / P(X)$$

Bayesian Belief Network

Bayesian Belief Networks specify joint conditional probability distributions. They are also known as Belief Networks, Bayesian Networks, or Probabilistic Networks.

- A Belief Network allows class conditional independencies to be defined between subsets of variables.
- It provides a graphical model of causal relationship on which learning can be performed.
- We can use a trained Bayesian Network for classification.

There are two components that define a Bayesian Belief Network –

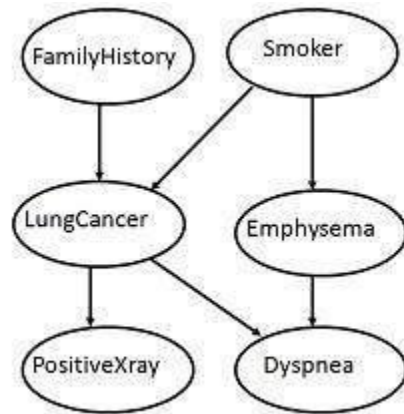
- Directed acyclic graph
- A set of conditional probability tables

Directed Acyclic Graph

- Each node in a directed acyclic graph represents a random variable.
- These variable may be discrete or continuous valued.
- These variables may correspond to the actual attribute given in the data.

Directed Acyclic Graph Representation

The following diagram shows a directed acyclic graph for six Boolean variables.



The arc in the diagram allows representation of causal knowledge. For example, lung cancer is influenced by a person's family history of lung cancer, as well as whether or not the person is a smoker. It is worth noting that the variable PositiveXray is independent of whether the patient has a family history of lung cancer or that the patient is a smoker, given that we know the patient has lung cancer.

Conditional Probability Table

The conditional probability table for the values of the variable LungCancer (LC) showing each possible combination of the values of its parent nodes, FamilyHistory (FH), and Smoker (S) is as follows –

	FH,S	FH,-S	-FH,S	-FH,-S
LC	0.8	0.5	0.7	0.1
-LC	0.2	0.5	0.3	0.9

Rule Based Classification –

IF-THEN Rules

Rule-based classifier makes use of a set of IF-THEN rules for classification. We can express a rule in the following form –

IF condition THEN conclusion

Let us consider a rule R1,

R1: IF age = youth AND student = yes
THEN buy_computer = yes

Points to remember –

- The IF part of the rule is called **rule antecedent** or **precondition**.
- The THEN part of the rule is called **rule consequent**.
- The antecedent part the condition consist of one or more attribute tests and these tests are logically ANDed.
- The consequent part consists of class prediction.

Note – We can also write rule R1 as follows –

R1: (age = youth) $\wedge$ (student = yes)(buys computer = yes)

If the condition holds true for a given tuple, then the antecedent is satisfied.

Rule Extraction

Here we will learn how to build a rule-based classifier by extracting IF-THEN rules from a decision tree.

Points to remember –

To extract a rule from a decision tree –

- One rule is created for each path from the root to the leaf node.
- To form a rule antecedent, each splitting criterion is logically ANDed.
- The leaf node holds the class prediction, forming the rule consequent.

Rule Induction Using Sequential Covering Algorithm

Sequential Covering Algorithm can be used to extract IF-THEN rules form the training data. We do not require to generate a decision tree first. In this algorithm, each rule for a given class covers many of the tuples of that class.

Some of the sequential Covering Algorithms are AQ, CN2, and RIPPER. As per the general strategy the rules are learned one at a time. For each time rules are learned, a tuple covered by the rule is removed and the process continues for the rest of the tuples. This is because the path to each leaf in a decision tree corresponds to a rule.

Note – The Decision tree induction can be considered as learning a set of rules simultaneously.

The Following is the sequential learning Algorithm where rules are learned for one class at a time. When learning a rule from a class C_i , we want the rule to cover all the tuples from class C only and no tuple form any other class.

Algorithm: Sequential Covering

Input:

D, a data set class-labeled tuples,

Att_vals, the set of all attributes and their possible values.

Output: A Set of IF-THEN rules.

Method:

Rule_set={ }; // initial set of rules learned is empty

for each class c do

 repeat

 Rule = Learn_One_Rule(D, Att_vals, c);

 remove tuples covered by Rule from D;

 until termination condition;

 Rule_set=Rule_set+Rule; // add a new rule to rule-set

end for

return Rule_Set;

Rule Pruning

The rule is pruned is due to the following reason –

- The Assessment of quality is made on the original set of training data. The rule may perform well on training data but less well on subsequent data. That's why the rule pruning is required.
- The rule is pruned by removing conjunct. The rule R is pruned, if pruned version of R has greater quality than what was assessed on an independent set of tuples.

FOIL is one of the simple and effective method for rule pruning. For a given rule R,

$FOIL_Prune = pos - neg / pos + neg$

where pos and neg is the number of positive tuples covered by R, respectively.

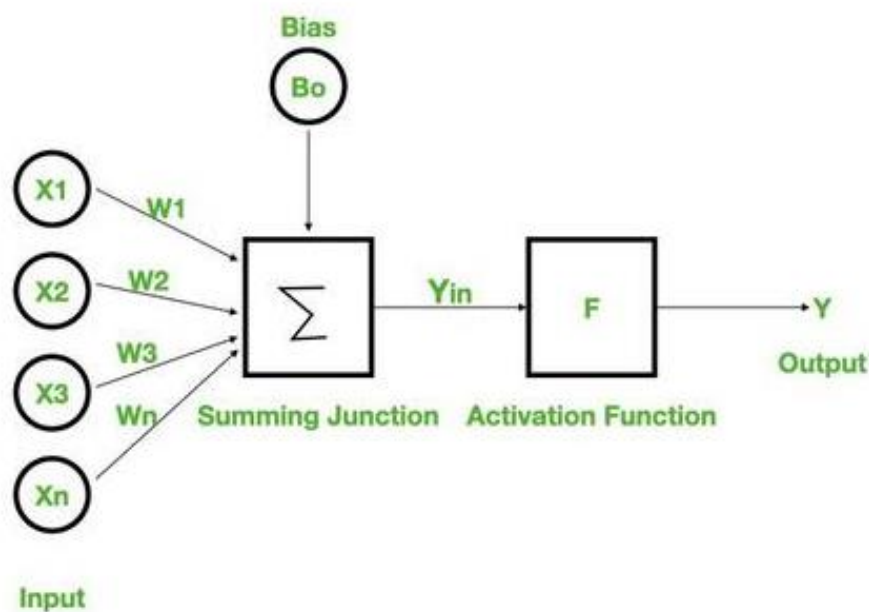
Note – This value will increase with the accuracy of R on the pruning set. Hence, if the FOIL_Prune value is higher for the pruned version of R, then we prune R.

Classification by Back propagation –

Backpropagation is an algorithm that backpropagates the errors from the output nodes to the input nodes. Therefore, it is simply referred to as the backward propagation of errors. It uses in the vast applications of neural networks in data mining like Character recognition, Signature verification, etc.

Neural Network:

Neural networks are an information processing paradigm inspired by the human nervous system. Just like in the human nervous system, we have biological neurons in the same way in neural networks we have artificial neurons, artificial neurons are mathematical functions derived from biological neurons. The human brain is estimated to have about 10 billion neurons, each connected to an average of 10,000 other neurons. Each neuron receives a signal through a synapse, which controls the effect of the signal concerning the neuron.

**Backpropagation:**

Backpropagation is a widely used algorithm for training feed forward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Features of Backpropagation:

1. it is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.

2. it is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
3. training is done in the three stages :
 - the feed-forward of input training pattern
 - the calculation and backpropagation of the error
 - updation of the weight

Working of Backpropagation:

Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

Backpropagation Algorithm:

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

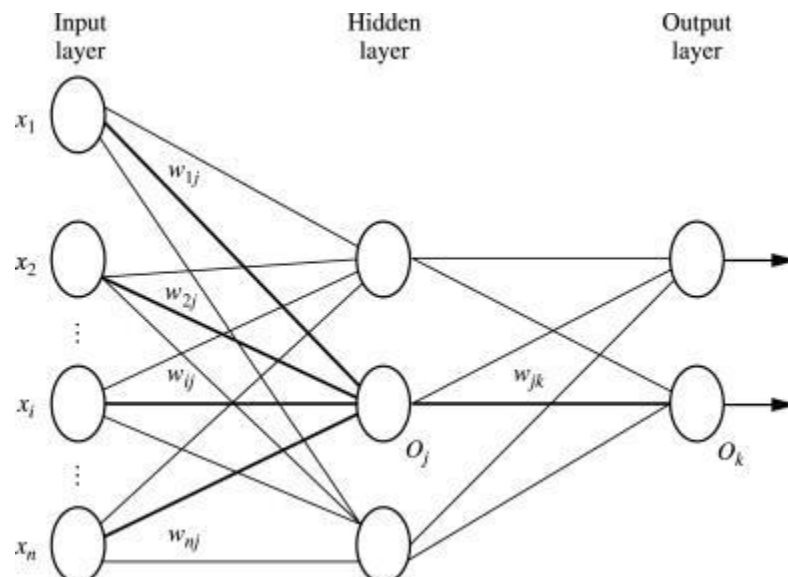
Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs

Backpropagation Error= Actual Output – Desired Output

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Parameters :

- x = inputs training vector $x=(x_1, x_2, \dots, x_n)$.
- t = target vector $t=(t_1, t_2, \dots, t_n)$.
- δ_k = error at output unit.
- δ_j = error at hidden layer.
- α = learning rate.
- V_{0j} = bias of hidden unit j .

Training Algorithm :

Step 1: Initialize weight to small random values.

Step 2: While the steps stopping condition is to be false do step 3 to 10.

Step 3: For each training pair do step 4 to 9 (Feed-Forward).

Step 4: Each input unit receives the signal unit and transmits the signal x_i signal to all the units.

Step 5 : Each hidden unit Z_j ($j=1$ to a) sums its weighted input signal to calculate its net input

$$z_{inj} = v_{0j} + \sum x_i v_{ij} \quad (i=1 \text{ to } n)$$

Applying activation function $z_j = f(z_{inj})$ and sends this signals to all units in the layer about i.e output units

For each output l =unit y_k ($k=1$ to m) sums its weighted input signals.

$$y_{ink} = w_{0k} + \sum z_i w_{jk} \quad (j=1 \text{ to } a)$$

and applies its activation function to calculate the output signals.

$$y_k = f(y_{ink})$$

Backpropagation Error :

Step 6: Each output unit y_k ($k=1$ to n) receives a target pattern corresponding to an input pattern then error is calculated as:

$$\delta_k = (t_k - y_k) + y_{ink}$$

Step 7: Each hidden unit Z_j ($j=1$ to a) sums its input from all units in the layer above

$$\delta_{inj} = \sum \delta_j w_{jk}$$

The error information term is calculated as :

$$\delta_j = \delta_{inj} + z_{inj}$$

Updation of weight and bias :

Step 8: Each output unit y_k ($k=1$ to m) updates its bias and weight ($j=1$ to a). The weight correction term is given by :

$$\Delta w_{jk} = \alpha \delta_k z_j$$

and the bias correction term is given by $\Delta w_k = \alpha \delta_k$.

therefore $w_{jk(\text{new})} = w_{jk(\text{old})} + \Delta w_{jk}$

$$w_{0k(\text{new})} = w_{0k(\text{old})} + \Delta w_{0k}$$

for each hidden unit z_j ($j=1$ to a) update its bias and weights ($i=0$ to n) the weight connection term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

and the bias connection on term

$$\Delta v_{0j} = \alpha \delta_j$$

Therefore $v_{ij(\text{new})} = v_{ij(\text{old})} + \Delta v_{ij}$

$$v_{0j(\text{new})} = v_{0j(\text{old})} + \Delta v_{0j}$$

Step 9: Test the stopping condition. The stopping condition can be the minimization of error, number of epochs.

Need for Backpropagation:

Backpropagation is “backpropagation of errors” and is very useful for training neural networks. It’s fast, easy to implement, and simple. Backpropagation does not require any parameters to be set, except the number of inputs. Backpropagation is a flexible method because no prior knowledge of the network is required.

Types of Backpropagation

There are two types of backpropagation networks.

- **Static backpropagation:** Static backpropagation is a network designed to map static inputs for static outputs. These types of networks are capable of solving static classification problems such as OCR (Optical Character Recognition).
- **Recurrent backpropagation:** Recursive backpropagation is another network used for fixed-point learning. Activation in recurrent backpropagation is feed-forward until a fixed value is reached. Static backpropagation provides an instant mapping, while recurrent backpropagation does not provide an instant mapping.

Advantages:

- It is simple, fast, and easy to program.
- Only numbers of the input are tuned, not any other parameter.
- It is Flexible and efficient.
- No need for users to learn any special functions.

Disadvantages:

- It is sensitive to noisy data and irregularities. Noisy data can lead to inaccurate results.
- Performance is highly dependent on input data.

- Spending too much time training.
- The matrix-based approach is preferred over a mini-batch.

Support Vector Machines –

Associative Classification –

Data mining is the process of discovering and extracting hidden patterns from different types of data to help decision-makers make decisions. Associative classification is a common classification learning method in data mining, which applies association rule detection methods and classification to create classification models.

Association Rule learning in Data Mining:

Association rule learning is a machine learning method for discovering interesting relationships between variables in large databases. It is designed to detect strong rules in the database based on some interesting metrics. For any given multi-item transaction, association rules aim to obtain rules that determine how or why certain items are linked.

Association rules are created by searching for information on common if-then patterns and using specific criteria with support and trust to define what the key relationships are. They help to show the frequency of an item in a given data since confidence is defined by the number of times an if-then statement is found to be true. However, a third criterion called lift is often used to compare expected and actual confidence. Lift shows how many times the if-then statement was predicted to be true. Create association rules to compute itemsets based on data created by two or more items. Association rules usually consist of rules that are well represented by the data.

There are different types of data mining techniques that can be used to find out the specific analysis and result like Classification analysis, Clustering analysis, and multivariate analysis. Association rules are mainly used to analyze and predict customer behavior.

- In Classification analysis, it is mostly used to question, make decisions, and predict behavior.
- In Clustering analysis, it is mainly used when no assumptions are made about possible relationships in the data.
- In Regression analysis, it is used when we want to predict an infinitely dependent value of a set of independent variables.

How does Association Rule Learning work?

Association rule learning is a type of unsupervised learning technique that checks for the dependency of one data item on another data item and maps accordingly so that it can be more profitable. It is based on different rules to discover the interesting relations between variables in the database. The

association rule learning is one of the very important concepts of machine learning, and it is employed in Market Basket analysis, Web usage mining, continuous production, etc. Here market basket analysis is a technique used by the various big retailer to discover the associations between items.

Association rule learning works on the concept of If and Else Statement, such as if A then B.

Here the If element is called antecedent, and then statement is called as Consequent.

These types of relationships where we can find out some association or relation between two items is known as single cardinality. It is all about creating rules, and if the number of items increases, then cardinality also increases accordingly. So, to measure the associations between thousands of data items, there are several metrics. These metrics are given below:

- Support
- Confidence
- Lift

1.Support :

Support is the frequency of A or how frequently an item appears in the dataset. It is defined as the fraction of the transaction T that contains the itemset X. If there are X datasets, then for transactions T, it can be written as:

$$\text{Supp}(X) = \text{Freq}(X) / T$$

2.Confidence:

Confidence indicates how often the rule has been found to be true. Or how often the items X and Y occur together in the dataset when the occurrence of X is already given. It is the ratio of the transaction that contains X and Y to the number of records that contain X.

$$\text{Confidence} = \text{Freq}(X, Y) / \text{Freq}(X)$$

3.Lift:

It is the strength of any rule, which can be defined as below formula: It is the ratio of the observed support measure and expected support if X and Y are independent of each other. It has three possible values:

$$\text{Lift} = \text{Supp}(X, Y) / \text{Supp}(X) * \text{Supp}(Y)$$

- If Lift= 1: The probability of occurrence of antecedent and consequent is independent of each other.
- Lift>1: It determines the degree to which the two itemsets are dependent to each other.
- Lift<1: It tells us that one item is a substitute for other items, which means one item has a negative effect on another.

Types of Association Rule Learning:

Association rule learning can be divided into three algorithms:

1.Apriori Algorithm:

This algorithm uses frequent datasets to generate association rules. It is designed to work on the databases that contain transactions. This algorithm uses a breadth-first search and Hash Tree to calculate the itemset efficiently. It is mainly used for market basket analysis and helps to understand the products that can be bought together. It can also be used in the healthcare field to find drug reactions for patients.

2.Eclat Algorithm:

Eclat algorithm stands for Equivalence Class Transformation. This algorithm uses a depthfirst search technique to find frequent itemsets in a transaction database. It performs faster execution than Apriori Algorithm.

3.F-P Growth Algorithm:

The F-P growth algorithm stands for Frequent Pattern, and it is the improved version of the Apriori Algorithm. It represents the database in the form of a tree structure that is known as a frequent pattern or tree. The purpose of this frequent tree is to extract the most frequent pattern.

Applications of Association Rule Learning:

It has various applications in machine learning and data mining. Below are some popular applications of association rule learning:

- **Market Basket Analysis:** It is one of the popular examples and applications of association rule mining. This technique is commonly used by big retailers to determine the association between items.
- **Medical Diagnosis:** With the help of association rules, patients can be cured easily, as it helps in identifying the probability of illness for a particular disease.
- **Protein Sequence:** The association rules help in determining the synthesis of artificial Proteins.
- It is also used for the **Catalog Design** and **Loss-leader Analysis** and many more other applications.

Associative Classification in Data Mining:

Bing Liu Et Al was the first to propose associative classification, in which he defined a model whose rule is “the right-hand side is constrained to be the attribute of the classification class”.An associative classifier is a supervised learning model that uses association rules to assign a target value.

The model generated by the association classifier and used to label new records consists of association rules that produce class labels. Therefore, they can also be thought of as a list of “if-then”

clauses: if a record meets certain criteria (specified on the left side of the rule, also known as antecedents), it is marked (or scored) according to the rule's category on the right. Most associative classifiers read the list of rules sequentially and apply the first matching rule to mark new records. Association classifier rules inherit some metrics from association rules, such as Support or Confidence, which can be used to rank or filter the rules in the model and evaluate their quality.

Types of Associative Classification:

There are different types of Associative Classification Methods, Some of them are given below.

1. CBA (Classification Based on Associations): It uses association rule techniques to classify data, which proves to be more accurate than traditional classification techniques. It has to face the sensitivity of the minimum support threshold. When a lower minimum support threshold is specified, a large number of rules are generated.

2. CMAR (Classification based on Multiple Association Rules): It uses an efficient FP-tree, which consumes less memory and space compared to Classification Based on Associations. The FP-tree will not always fit in the main memory, especially when the number of attributes is large.

3. CPAR (Classification based on Predictive Association Rules): Classification based on predictive association rules combines the advantages of association classification and traditional rule-based classification. Classification based on predictive association rules uses a greedy algorithm to generate rules directly from training data. Furthermore, classification based on predictive association rules generates and tests more rules than traditional rule-based classifiers to avoid missing important rules.

Lazy Learners –

Classification may be defined as the process of predicting class or category from observed values or given data points. The categorized output can have the form such as “Black” or “White” or “spam” or “no spam”.

Mathematically, classification is the task of approximating a mapping function (f) from input variables (X) to output variables (Y). It basically belongs to the supervised machine learning in which targets are also provided along with the input data set.

An example of classification problem can be the spam detection in emails. There can be only two categories of output, “spam” and “no spam”; hence this is a binary type classification.

To implement this classification, we first need to train the classifier. For this example, “spam” and “no spam” emails would be used as the training data. After successfully train the classifier, it can be used to detect an unknown email.

Types of Learners in Classification

We have two types of learners in respective to classification problems –

Lazy Learners

As the name suggests, such kind of learners waits for the testing data to be appeared after storing the training data. Classification is done only after getting the testing data. They spend less time on training but more time on predicting. Examples of lazy learners are K-nearest neighbor and case-based reasoning.

Eager Learners

As opposite to lazy learners, eager learners construct classification model without waiting for the testing data to be appeared after storing the training data. They spend more time on training but less time on predicting. Examples of eager learners are Decision Trees, Naïve Bayes and Artificial Neural Networks (ANN).

Lazy learning is a type of machine learning that doesn't process training data until it needs to make a prediction. Instead of building models during training, lazy learning algorithms wait until they encounter a new query. This method stores and compares training examples when making predictions. It's also called instance-based or memory-based learning.

Lazy Learning Explained

Lazy learning algorithms work by memorizing the training data rather than constructing a general model. When a new query is received, lazy learning retrieves similar instances from the training set and uses them to generate a prediction. The similarity between instances is usually calculated using distance metrics, such as Euclidean distance or cosine similarity.

One of the most popular lazy learning algorithms is the [k-nearest neighbors](#) (k-NN) algorithm. In k-NN, the k closest training instances to the query point are considered, and their class labels are used to determine the class of the query. Lazy learning methods excel in situations where the underlying data distribution is complex or where the training data is noisy.

Examples of Real-World Lazy Learning Applications

Lazy learning has found applications in various domains. Here are a few examples:

- **Recommendation systems.** Lazy learning is widely used in recommender systems to provide personalized recommendations. By comparing user preferences to similar users in the training set,

lazy learning algorithms can suggest items or products of interest, such as movies, books, or products.

- **Medical diagnosis.** Lazy learning can be employed in medical diagnosis systems. By comparing patient symptoms and medical histories to similar cases in the training data, lazy learning algorithms can assist in diagnosing diseases or suggesting appropriate treatments.
- **Anomaly detection.** Lazy learning algorithms are useful for detecting anomalies or outliers in datasets. For example, an algorithm can detect credit card fraud by comparing a transaction to nearby transactions based on factors like location and history. If the transaction is unusual, such as being made in a faraway location for a large amount, it may be flagged as fraudulent.

Lazy Learning vs Eager Learning Models

Lazy learning stands in contrast to eager learning methods, such as decision trees or neural networks, where models are built during the training phase. Here are some key differences:

- **Training phase.** Eager learning algorithms construct a general model based on the entire training dataset, whereas lazy learning algorithms defer model construction until prediction time.
- **Computational cost.** Lazy learning algorithms can be computationally expensive during prediction since they require searching through the training data to find nearest neighbors. In contrast, eager learning algorithms typically have faster prediction times once the model is trained.
- **Interpretability.** Eager learning methods often provide more interpretability as they produce explicit models, such as decision trees, that can be easily understood by humans. Lazy learning methods, on the other hand, rely on the stored instances and do not provide explicit rules or models.

What are the Benefits of Lazy Learning?

Lazy learning offers several advantages:

- **Adaptability.** Lazy learning algorithms can adapt quickly to new or changing data. Since the learning process happens at prediction time, they can incorporate new instances without requiring complete retraining of the model.
- **Robustness to outliers.** Lazy learning algorithms are less affected by outliers compared to eager learning methods. Outliers have less influence on predictions because they are not used during the learning phase.
- **Flexibility.** When it comes to handling complex data distributions and nonlinear relationships, lazy learning algorithms are effective. They can capture intricate decision boundaries by leveraging the information stored in the training instances.

What are the Limitations of Lazy Learning?

Despite its benefits, lazy learning has certain limitations that should be considered:

- **High prediction time.** Lazy learning can be slower at prediction time compared to eager learning methods. Since they require searching through the training data to find nearest neighbors, the computational cost can be significant, especially with large datasets.
- **Storage requirements.** Lazy learning algorithms need to store the entire training dataset or a representative subset of it. This can be memory-intensive, particularly when dealing with large datasets with high-dimensional features.
- **Sensitivity to noise.** Noise or irrelevant features in the training data can significantly impact the accuracy of lazy learning model predictions, because they rely on direct comparison with stored instances.
- **Overfitting.** Lazy learning algorithms are prone to overfitting when the training dataset is small or when there are too many stored instances. Overfitting occurs when the model memorizes the training instances, including their noise or outliers, leading to poor generalization on unseen data.
- **Lack of transparency.** Lazy learning methods do not provide explicit models or rules that can be easily interpreted. This lack of transparency makes it challenging to understand the reasoning behind specific predictions or to extract actionable insights from the model.

Other Classification Methods – Prediction –

There are two forms of data analysis that can be used for extracting models describing important classes or to predict future data trends. These two forms are as follows –

- **Classification**
- **Prediction**

Classification models predict categorical class labels; and prediction models predict continuous valued functions. For example, we can build a classification model to categorize bank loan applications as either safe or risky, or a prediction model to predict the expenditures in dollars of potential customers on computer equipment given their income and occupation.

What is classification?

Following are the examples of cases where the data analysis task is Classification –

- A bank loan officer wants to analyze the data in order to know which customer (loan applicant) are risky or which are safe.
- A marketing manager at a company needs to analyze a customer with a given profile, who will buy a new computer.

In both of the above examples, a model or classifier is constructed to predict the categorical labels. These labels are risky or safe for loan application data and yes or no for marketing data.

What is prediction?

Following are the examples of cases where the data analysis task is Prediction –

Suppose the marketing manager needs to predict how much a given customer will spend during a sale at his company. In this example we are bothered to predict a numeric value. Therefore the data analysis task is an example of numeric prediction. In this case, a model or a predictor will be constructed that predicts a continuous-valued-function or ordered value.

Note – Regression analysis is a statistical methodology that is most often used for numeric prediction.

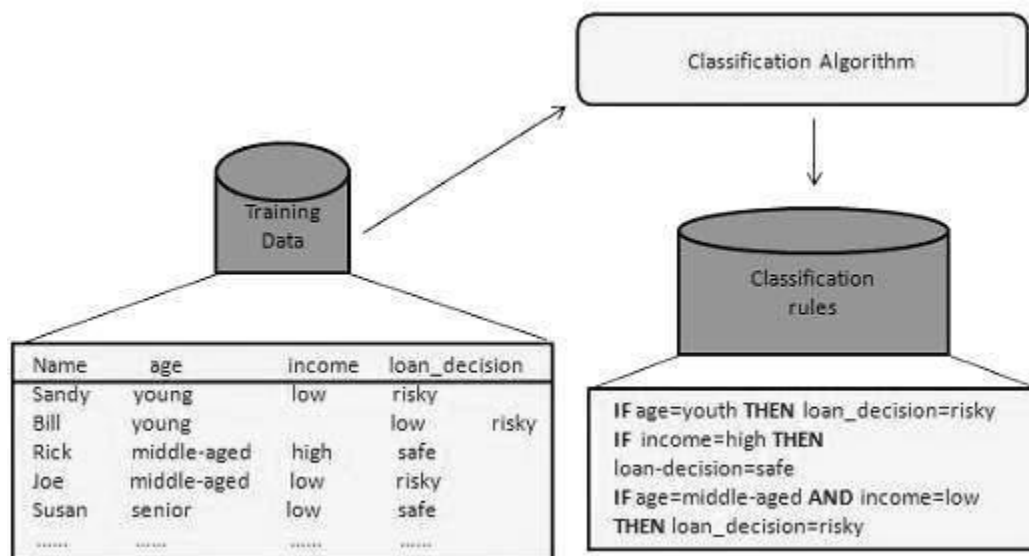
How Does Classification Works?

With the help of the bank loan application that we have discussed above, let us understand the working of classification. The Data Classification process includes two steps –

- Building the Classifier or Model
- Using Classifier for Classification

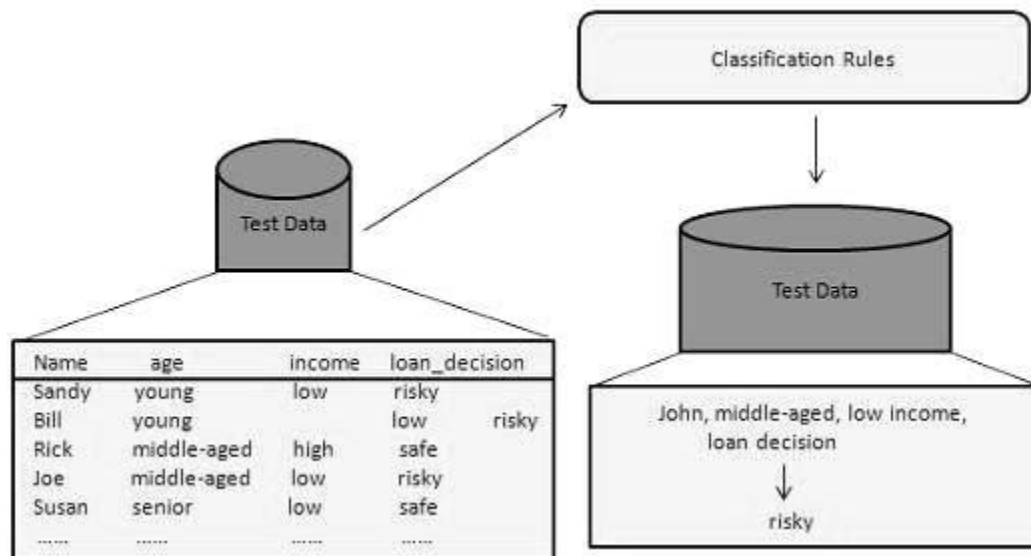
Building the Classifier or Model

- This step is the learning step or the learning phase.
- In this step the classification algorithms build the classifier.
- The classifier is built from the training set made up of database tuples and their associated class labels.
- Each tuple that constitutes the training set is referred to as a category or class. These tuples can also be referred to as sample, object or data points.



Using Classifier for Classification

In this step, the classifier is used for classification. Here the test data is used to estimate the accuracy of classification rules. The classification rules can be applied to the new data tuples if the accuracy is considered acceptable.

**Classification and Prediction Issues**

The major issue is preparing the data for Classification and Prediction. Preparing the data involves the following activities –

- **Data Cleaning** – Data cleaning involves removing the noise and treatment of missing values. The noise is removed by applying smoothing techniques and the problem of missing values is solved by replacing a missing value with most commonly occurring value for that attribute.
- **Relevance Analysis** – Database may also have the irrelevant attributes. Correlation analysis is used to know whether any two given attributes are related.
- **Data Transformation and reduction** – The data can be transformed by any of the following methods.
 - **Normalization** – The data is transformed using normalization. Normalization involves scaling all values for given attribute in order to make them fall within a small specified range. Normalization is used when in the learning step, the neural networks or the methods involving measurements are used.
 - **Generalization** – The data can also be transformed by generalizing it to the higher concept. For this purpose we can use the concept hierarchies.

Note – Data can also be reduced by some other methods such as wavelet transformation, binning, histogram analysis, and clustering.

Comparison of Classification and Prediction Methods

Here is the criteria for comparing the methods of Classification and Prediction –

- **Accuracy** – Accuracy of classifier refers to the ability of classifier. It predict the class label correctly and the accuracy of the predictor refers to how well a given predictor can guess the value of predicted attribute for a new data.
- **Speed** – This refers to the computational cost in generating and using the classifier or predictor.
- **Robustness** – It refers to the ability of classifier or predictor to make correct predictions from given noisy data.
- **Scalability** – Scalability refers to the ability to construct the classifier or predictor efficiently; given large amount of data.
- **Interpretability** – It refers to what extent the classifier or predictor understands.

Accuracy and Error Measures –

Evaluating the Accuracy of a Classifier or Predictor –

Classifier Accuracy

Evaluating & estimating the accuracy of classifiers is important in that it allows one to evaluate how accurately a given classifier will label future data, that, is, data on which the classifier has not been trained.

For example, suppose you used data from previous sales to train a classifier to predict customer purchasing behavior.

You would like an estimate of how accurately the classifier can predict the purchasing behavior of future customers, that is, future customer data on which the classifier has not been trained.

Accuracy estimates to help in the comparison of different classifiers.

Methods To Find Accuracy Of The Classifiers

- Holdout Method
- Random Subsampling
- K-fold Cross-Validation
- Bootstrap Methods

Estimating The Classifier Accuracy

Holdout & Random Subsampling

The **holdout method** is what we have alluded to so far in our discussions about accuracy.

In this method, the given data are randomly partitioned into two independent sets, a training set, and a test set.

Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set.

The training set is used to derive the model, whose accuracy is estimated with the test set.

The estimate is pessimistic because only a portion of the initial data is used to derive the model.

Random subsampling is a variation of the holdout method in which the holdout method is repeated k times.

The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration. (For prediction, we can take the average of the predictor error rates.)

Cross-Validation

In **k-fold cross-validation**, the initial data are randomly partitioned into k mutually exclusive subsets or “folds,” $D_1, D_2, \dots, D_k$, each of approximately equal size.

Training and testing are performed k times. In iteration i , partition D_i is reserved as the test set, and the remaining partitions are collectively used to train the model.

That is, in the first iteration, subsets $D_2, \dots, D_k$ collectively serves as the training set to obtain a first model, which is tested on D_1 ; the second iteration is trained on subsets $D_1, D_3, \dots, D_k$ and tested on D_2 ; and so on.

Unlike the holdout and random subsampling methods above, here, each sample is used the same number of times for training and once for testing.

For classification, the accuracy estimate is the overall number of correct classifications from the k iterations, divided by the total number of tuples in the initial data.

For prediction, the error estimate can be computed as the total loss from the k iterations, divided by the total number of initial tuples.

Bootstrapping

Unlike the accuracy estimation methods mentioned above, the bootstrap method samples the given training tuples uniformly with replacement.

That is, each time a tuple is selected, it is equally likely to be selected again and readded to the training set.

For instance, imagine a machine that randomly selects tuples for our training set. In sampling with replacement, the machine is allowed to select the same tuple more than once.

Ensemble Methods –

Ensemble Methods - Increasing The Accuracy

Are there general strategies for improving classifier and predictor accuracy?

YES, Bagging and boosting are two such techniques.

Bagging

We first take an intuitive look at how bagging works as a method of increasing accuracy.

For ease of explanation, we will assume at first that our model is a classifier. Suppose that you are a patient and would like to have a diagnosis made based on your symptoms.

Instead of asking one doctor, you may choose to ask several. If a certain diagnosis occurs more than any of the others, you may choose this as the final or best diagnosis.

That is, the final diagnosis is made based on a majority vote, where each doctor gets an equal vote. Now replace each doctor by a classifier, and you have the basic idea behind bagging.

Intuitively, a majority vote made by a large group of doctors may be more reliable than a majority vote made by a small group.

Given a set, D , of d tuples, bagging works as follows.

For iteration i ($i = 1, 2, \dots, k$), a training set, D_i , of d tuples is sampled with replacement from the original set of tuples, D .

Note that the term bagging stands for bootstrap aggregation.

A classifier model, M_i , is learned for each training set, D_i .

To classify an unknown tuple, X , each classifier, M_i , returns its class prediction, which counts as one vote.

The bagged classifier, M , counts the votes and assigns the class with the most votes to X .

Bagging can be applied to the prediction of continuous values by taking the average value of each prediction for a given test tuple.

The bagged classifier often has significantly greater accuracy than a single classifier derived from D , the original training data.

Boosting

We now look at the ensemble method of boosting. As in the previous section, suppose that as a patient, you have certain symptoms.

Instead of consulting one doctor, you choose to consult several.

Suppose you assign weights to the value or worth of each doctor's diagnosis, based on the accuracies of previous diagnoses they have made.

The final diagnosis is then a combination of the weighted diagnoses. This is the essence behind boosting.

In boosting, weights are assigned to each training tuple.

A series of k classifiers is iteratively learned. After a classifier M_i is learned, the weights are updated to allow the subsequent classifier, M_{i+1} , to “pay more attention” to the training tuples that were misclassified by M_i .

The final boosted classifier, M , combines the votes of each individual classifier, where the weight of each classifier’s vote is a function of its accuracy.

The boosting algorithm can be extended for the prediction of continuous values.

Model Section.

Introduction to Models in Data Mining

Data Mining uses raw data to extract information or, in fact, mining the required information from data. Data Mining is used in the most diverse range of applications, including political model forecasting, weather pattern model forecasting, website ranking forecasting, etc. Data mining is also used in organizations that use big data as their raw data source to mine the required data, which can be quite complex.

Techniques Used in Data Mining

Data Mining model is created by applying the algorithm on top of the raw data. The mining model is more than the algorithm or metadata handler. It is a set of data, patterns, statistics that can be serviceable on new data that is being sourced to generate the predictions and get some inference about the relationships. The following are some of the techniques that are used in data mining.

1. Descriptive Data Mining Technique

This technique is generally preferred to generate cross-tabulation, correlation, frequency, etc. These descriptive data mining techniques are used to obtain information on the data’s regularity by using raw data as input and discovering important patterns. The other application of this is to understand the captivating groups in the wider area of the raw data.

2. Predictive Data Mining Technique

The main objective of the predictive mining technique is to identify futuristic results instead of the current tendency. Many functions are used for the prediction of the target value. The techniques that fall under this category are classification, regression and time-series analysis. Data modelling is a

compulsion for this predictive analysis, which uses some variables to predict the uncertain futuristic data for other variables.

Types of Models in Data Mining

A few of the data mining models are mentioned below, along with their description:

1. Fraud Claiming Models

Fraud is the challenge faced by many industries and especially the insurance industry. These industries need to constantly predict using the raw data so that the fraud claims can be understood and acted upon. We can track the claims that arrive in the form of raw data and identify the likelihood of it being fraudulent, resulting in large savings for the insurance company.

2. Customer Clone Models

The customer clone model can predict which prospects are highly likely to respond based on the characteristics of the organization's "best customers".

3. Response Models

Predictive data mining response models help organisations identify the usage patterns that segregate their customer base to establish contact with those customers. This response model is the best method for predicting and identifying the customer base or prospects to the target for a particular product. The offering is in line with the use of a model developed. These models are applied in identifying the customers who are highly likely to possess the characteristic of being targeted.

4. Revenue and Profit Predictive Models

Revenue and Profit Prediction models combine the response or nonresponse characteristics with a given revenue estimate, especially if ordered sizes, margins are differing widely or monthly billings. As we know, not all the responses have the same or equal value, and the model that can increase the responses doesn't necessarily gain our profit. Revenue and profit predictive technique indicate that those respondents who are highly likely to increase revenue or profit delta margin with their response than the other responders. These are some of the model types, and there are many more that can help mining the required data from the set of raw data.
